

在 Dataproc 叢集中執行 Hadoop 字數計算工作

來源：<https://codelabs.developers.google.com/codelabs/intro-cloud-composer?hl=zh-tw>

步驟數：7

本程式碼實驗室將說明如何使用 Cloud Composer 建立及執行 Apache Airflow 工作流程，完成下列工作：建立 Cloud Dataproc 叢集、在叢集上執行 Apache Hadoop 字數統計工作，並將結果輸出至 Cloud Storage，以及刪除叢集。本頁面也會說明如何透過 Google Cloud Platform 主控台、Cloud SDK 和 Airflow 網頁介面存取 Cloud Composer 環境。

目錄

1. [1. 簡介](#)
2. [2. 設定 GCP](#)
3. [3. 設定 Apache Airflow](#)
4. [4. 工作流程範例](#)
5. [5. 將 Airflow 檔案上傳至 Cloud Storage](#)
6. [6. 使用 Airflow UI](#)
7. [7. 清除所用資源](#)

1. 簡介

工作流程是資料分析的常見用途，包括擷取、轉換及分析資料，找出其中的重要資訊。在 Google Cloud Platform 中，工作流程自動化調度管理工具是 Cloud Composer，這是熱門開放原始碼工作流程工具 Apache Airflow 的代管版本。在本實驗室中，您將使用 Cloud Composer 建立簡單的工作流程，該工作流程會建立 Cloud Dataproc 叢集、使用 Cloud Dataproc 和 Apache Hadoop 分析叢集，然後刪除 Cloud Dataproc 叢集。

什麼是 Cloud Composer ?

Cloud Composer 是一項全代管工作流程自動化調度管理服務，能讓您建立、排程和監控散布於不同雲端和內部部署資料中心的管道。Cloud Composer 以熱門的 **Apache Airflow** 開放原始碼專案為基礎，並採用 Python 程式設計語言，不僅相當容易使用，也能讓您免受單一架構限制。

使用 Cloud Composer 而非 Apache Airflow 的本機執行個體，使用者可享有 Airflow 的最佳功能，且不必安裝或管理。

如要進一步瞭解 Cloud Composer，請參閱[說明文件](#)！

什麼是 Apache Airflow ?

Apache Airflow 是一項開放原始碼工具，可用於以程式輔助方式建立、安排及監控工作流程。本實驗室會出現幾個與 Airflow 相關的重要詞彙，請務必記住：

- DAG：**DAG** (有向無環圖) 是您要排定及執行的工作集合。DAG (又稱工作流程) 定義於標準 Python 檔案中
- 運算子：運算子會說明工作流程中的單一工作

什麼是 Cloud Dataproc ?

Cloud Dataproc 是 Google Cloud Platform 的全代管 **Apache Spark** 和 **Apache Hadoop** 服務。Cloud Dataproc 可輕鬆與其他 GCP 服務整合，是功能強大且完善的平台，可用於資料處理、數據分析及機器學習。

如要進一步瞭解 Cloud Dataproc，請參閱[說明文件](#)！

學習內容

本程式碼研究室將說明如何建立及執行 Apache Airflow 工作流程，在 Cloud Composer 中完成下列工作：

- 建立 Cloud Dataproc 叢集
- 在叢集上執行 Apache Hadoop 字數計算工作，並將結果輸出至 Cloud Storage
- 刪除叢集

課程內容

- 如何在 Cloud Composer 中建立及執行 Apache Airflow 工作流程
- 如何使用 Cloud Composer 和 Cloud Dataproc 對資料集執行分析
- 如何透過 Google Cloud Platform 主控台、Cloud SDK 和 Airflow 網頁介面存取 Cloud Composer 環境

軟硬體需求

- GCP 帳戶
 - 具備 CLI 基本知識
 - 對 Python 有基本瞭解
-

步驟 2 · 約 5 分鐘

2. 設定 GCP

建立專案

選取或建立 Google Cloud Platform 專案。

記下專案 ID，後續步驟將會用到。

如要建立新專案，專案 ID 會顯示在建立頁面的「專案名稱」下方

New Project

Project Name *

My Project 303

Project ID: plucky-avatar-233921. It cannot be changed later. [EDIT](#)

Billing account *

My Billing Account

Any charges for this project will be billed to the account you select here.

Organization

google.com

This project will be attached to google.com.

Location *

[BROWSE](#)

Parent organization or folder

☐ Create Googleplex internal App Engine project ?

[CREATE](#)

[CANCEL](#)

如果已建立專案，您可以在[控制台首頁](#)的「專案資訊」資訊卡中找到 ID

[DASHBOARD](#) [ACTIVITY](#) [CUSTOMIZE](#)

Quick Access ([Feedback?](#))



Cloud Storage Browser

Accessed 2 minutes ago



Environments –
Composer – My Project...

Accessed 3 minutes ago



Composer – My Project
56570 – Google Cloud...

Accessed 3 minutes ago



Manage Resources

Accessed 1 hour ago



Project info

Project name

My Project 56570

Project ID

calcium-post-233919

Project number

765545603813

[Go to project settings](#)



Resources

This project has no resources



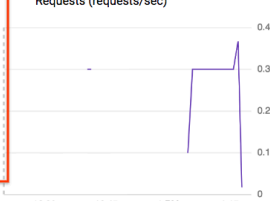
Trace

No trace data from the past 7 days

[Get started with Stackdriver Trace](#)

APIs

Requests (requests/sec)



Requests: 0.017

[Go to APIs overview](#)



Google Cloud Platform status

All services normal

[Go to Cloud status dashboard](#)

Billing

Estimated charges USD \$0.00

For the billing period Mar 1 – 8, 2019

[View detailed charges](#)



Error Reporting

No sign of any errors. Have you set up Error Reporting?

[Learn how to set up Error Reporting](#)

注意：如果您不打算保留在本教學課程中建立的資源，請建立新的專案，不要選擇現有的專案。這樣在課程結束後，您就可以刪除專案，移除與專案和教學課程相關聯的所有資源。

啟用 API

啟用 [Cloud Composer](#)、[Cloud Dataproc](#) 和 [Cloud Storage API](#)。啟用後，您可以忽略「前往憑證」按鈕，繼續進行本教學課程的下一個步驟。

Register your application for Cloud Composer API, Cloud Dataproc API, Cloud Storage in Google Cloud Platform

Google Cloud Platform allows you to manage your application and monitor API usage.

Select a project where your application will be registered

You can use one project to manage all of your applications, or you can create a different project for each application.

My Project 56570

Continue

Google Cloud Platform Select a project

The APIs are enabled

Cloud Composer API, Cloud Dataproc API, Cloud Storage have been enabled.

Next, to use the API you'll need the right credentials.

Go to credentials

如要進一步瞭解如何啟用及停用 API，請參閱[說明文件](#)。

建立 Composer 環境

建立 [Cloud Composer 環境](#)，並採用下列設定：

- 名稱：my-composer-environment
- 位置：us-central1
- 可用區：us-central1-a

其他設定則保留預設值。按一下底部的「建立」。



Composer

Environments

+ CREATE



Filter environments



Name ↑

Location

Creation time

Update t

No rows to display



Name *

my-composer-environment

Node configuration

The configuration information for the Google Kubernetes Engine nodes running the Airflow software.

Node count *

3

The number of nodes in the Google Kubernetes Engine cluster that will be used to run this environment.

Location *

us-central1

The Google Compute Engine region where the environment will be created.

Zone

us-central1-a

The Compute Engine zone in which to deploy the VMs used to run the Apache Airflow software. If unspecified, the service will pick a zone in the Compute Engine region corresponding to the selected location. [Learn more.](#)

如果在 GCP 主控台的「Environments」(環境) 頁面，環境名稱左側顯示綠色勾號，表示環境已建立完成。建立環境需要幾分鐘才能完成。在這段期間內，你仍可離開這個頁面。

建立 Cloud Storage bucket

在專案中建立 Cloud Storage bucket，並採用下列設定：

- 名稱：<your-project-id>
- 預設儲存空間級別：多區域
- 地點：美國
- 存取權控管模型：精細

準備就緒後，按下「建立」

Google Cloud Platform

Spark Jupyter Notebooks

Create a bucket

Name your bucket

Choose where to store your data

Choose a default storage class for your data

Choose how to control access to objects

Access control

Fine-grained

Specify access to individual objects by using object-level permissions (ACLs) in addition to your bucket-level permissions (IAM). [Learn more](#)

Uniform

Ensure uniform access to all objects in the bucket by using only bucket-level permissions (IAM). This option becomes permanent after 90 days. [Learn more](#)

CONTINUE

Advanced settings (optional)

CREATE

CANCEL

3. 設定 Apache Airflow

如果 Composer 環境尚未建立完成，請略過「範例工作流程」步驟，稍後再返回這個部分

查看 Composer 環境資訊

在 GCP 主控台中，開啟「Environments」(環境) 頁面。

按一下環境名稱即可查看詳細資料。

「環境詳細資料」頁面有各種資訊，例如 Airflow 網路介面網址、Google Kubernetes Engine 叢集 ID、Cloud Storage bucket 名稱，以及 /days 資料夾的路徑。

在 Airflow 中，**DAG** (有向無環圖) 是您要排程及執行的有組織工作集合。DAG (也稱為工作流程) 是在標準 Python 檔案中定義。Cloud Composer 只會為 /days 資料夾中的 DAG 排程。/days 資料夾位於 Cloud Composer 在您建立環境時自動建立的 Cloud Storage 值區中。

設定 Apache Airflow 環境變數

Apache Airflow 變數是 Airflow 的專屬概念，與**環境變數**不同。在本步驟中，您將設定下列三個 **Airflow 變數**：`gcp_project`、`gcs_bucket` 和 `gce_zone`。

使用 `gcloud` 設定變數

首先，請開啟 **Cloud Shell**，其中已安裝 Cloud SDK，方便您使用。

將環境變數 `COMPOSER_INSTANCE` 設為 Composer 環境的名稱

```
COMPOSER_INSTANCE=my-composer-environment
```

如要使用 `gcloud` 指令列工具設定 Airflow 變數，請使用 `gcloud composer environments run` 指令搭配 `variables` 子指令。這個 `gcloud composer` 指令會執行 Airflow CLI 子指令 `variables`。子指令會將引數傳遞至 `gcloud` 指令列工具。

您需要執行這項指令三次，並將變數替換為專案的相關變數。

使用下列指令設定 `gcp_project`，並將 `<your-project-id>` 替換為您步驟 2 中記下的專案 ID。

```
gcloud composer environments run ${COMPOSER_INSTANCE} \
  --location us-central1 variables -- --set gcp_project <your-project-id>
```

輸出內容應如下所示：

```
kubeconfig entry generated for us-central1-my-composer-env-123abc-gke.  
Executing within the following Kubernetes cluster namespace: composer-1-10-0-airflow-  
1-10-2-123abc  
[2020-04-17 20:42:49,713] {settings.py:176} INFO - settings.configure_orm(): Using  
pool settings. pool_size=5, pool_recycle=1800, pid=449  
[2020-04-17 20:42:50,123] {default_celery.py:90} WARNING - You have configured a  
result_backend of redis://airflow-redis-service.default.svc.cluste  
r.local:6379/0, it is highly recommended to use an alternative result_backend (i.e. a  
database).  
[2020-04-17 20:42:50,127] {__init__.py:51} INFO - Using executor CeleryExecutor  
[2020-04-17 20:42:50,433] {app.py:52} WARNING - Using default Composer Environment  
Variables. Overrides have not been applied.  
[2020-04-17 20:42:50,440] {configuration.py:522} INFO - Reading the config from  
/etc/airflow/airflow.cfg  
[2020-04-17 20:42:50,452] {configuration.py:522} INFO - Reading the config from  
/etc/airflow/airflow.cfg
```

使用下列指令設定 `gcs_bucket`，並將 `<your-bucket-name>` 替換為您步驟 2 中記下的 bucket ID。如果您按照建議操作，bucket 名稱會與專案 ID 相同。輸出內容會與先前的指令類似。

```
gcloud composer environments run ${COMPOSER_INSTANCE} \  
  --location us-central1 variables -- --set gcs_bucket gs://<your-bucket-name>
```

使用下列指令設定 `gce_zone`。輸出內容會與先前的指令類似。

```
gcloud composer environments run ${COMPOSER_INSTANCE} \  
  --location us-central1 variables -- --set gce_zone us-central1-a
```

(選用) 使用 `gcloud` 查看變數

如要查看變數的值，請透過 `get` 引數執行 Airflow CLI 子指令 `variables`，或使用 [Airflow 使用者介面](#)。

例如：

```
gcloud composer environments run ${COMPOSER_INSTANCE} \  
  --location us-central1 variables -- --get gcs_bucket
```

您可以使用剛才設定的三個變數 (`gcp_project`、`gcs_bucket` 和 `gce_zone`) 執行這項操作。

4. 工作流程範例

我們來看看步驟 5 中要使用的 DAG 程式碼。請先不要下載檔案，只要按照這裡的步驟操作即可。

這裡有很多需要注意的地方，以下將一一說明。

```
from airflow import models
from airflow.contrib.operators import dataproc_operator
from airflow.utils import trigger_rule
```

首先，請匯入一些 Airflow：

- `airflow.models` - 讓我們存取及建立 Airflow 資料庫中的資料。
- `airflow.contrib.operators` - 社群營運人員的居住地。在本例中，我們需要 `dataproc_operator` 才能存取 Cloud Dataproc API。
- `airflow.utils.trigger_rule` - For adding trigger rules to our operators. [觸發規則](#)可精細控管運算子是否應根據父項的狀態執行。

```
output_file = os.path.join(
    models.Variable.get('gcs_bucket'), 'wordcount',
    datetime.datetime.now().strftime('%Y%m%d-%H%M%S')) + os.sep
```

這會指定輸出檔案的位置。這裡值得注意的程式碼是 `models.Variable.get('gcs_bucket')`，這會從 Airflow 資料庫擷取 `gcs_bucket` 變數值。

```
WORDCOUNT_JAR = (
    'file:///usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar'
)

input_file = 'gs://pub/shakespeare/rose.txt'

wordcount_args = ['wordcount', input_file, output_file]
```

- `WORDCOUNT_JAR` - 我們最終會在 Cloud Dataproc 叢集上執行的 `.jar` 檔案位置。系統已為您代管在 GCP 上。
- `input_file` - 檔案位置，內含 Hadoop 工作最終會運算的資料。我們會在步驟 5 中，將資料一併上傳至該位置。
- `wordcount_args` - 我們將傳遞至 `jar` 檔案的引數。

```
yesterday = datetime.datetime.combine(
    datetime.datetime.today() - datetime.timedelta(1),
    datetime.datetime.min.time())
```

這會提供相當於前一天午夜的 `datetime` 物件。舉例來說，如果是在 3 月 4 日 11:00 執行這項作業，`datetime` 物件就會代表 3 月 3 日 00:00。這與 Airflow 的排程處理方式有關。詳情請參閱[這篇文章](#)。

```
default_dag_args = {
    'start_date': yesterday,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': datetime.timedelta(minutes=5),
    'project_id': models.Variable.get('gcp_project')
}
```

建立新的 DAG 時，應一律提供字典形式的 `default_dag_args` 變數：

- `'email_on_failure'` - 指出工作失敗時是否應發送電子郵件快訊
- `'email_on_retry'` - 指出在重試工作時是否應傳送電子郵件快訊
- `'retries'` - 表示 DAG 失敗時，Airflow 應重試的次數
- `'retry_delay'` - 表示 Airflow 應等待多久再嘗試重試
- `'project_id'` - 告知 DAG 要與哪個 GCP 專案 ID 建立關聯，後續使用 Dataproc 運算子時會需要這個 ID

```
with models.DAG(
    'composer_hadoop_tutorial',
    schedule_interval=datetime.timedelta(days=1),
    default_args=default_dag_args) as dag:
```

使用 `with models.DAG` 可讓指令碼將其下方的所有內容納入同一個 DAG。我們還看到傳遞的三個引數：

- 第一個是字串，也就是要建立的 DAG 名稱。在本例中，我們使用 `composer_hadoop_tutorial`。
- `schedule_interval` - `datetime.timedelta` 物件，在此我們已設為一天。這表示這個 DAG 會在 `'default_dag_args'` 中設定的 `'start_date'` 之後，每天嘗試執行一次
- `default_args` - 先前建立的字典，內含 DAG 的預設引數

建立 Dataproc 叢集

接著，我們會建立 `dataproc_operator.DataprocClusterCreateOperator`，這會建立 Cloud Dataproc 叢集。

```
create_dataproc_cluster = dataproc_operator.DataprocClusterCreateOperator(
    task_id='create_dataproc_cluster',
    cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
    num_workers=2,
    zone=models.Variable.get('gce_zone'),
    master_machine_type='n1-standard-1',
    worker_machine_type='n1-standard-1')
```

在這個運算子中，我們看到幾個引數，除了第一個引數外，其餘引數都專屬於這個運算子：

- `task_id` - 與 `BashOperator` 相同，這是我們指派給運算子的名稱，可從 Airflow UI 查看
- `cluster_name`：我們指派給 Cloud Dataproc 叢集的名稱。在此，我們將其命名為 `composer-hadoop-tutorial-cluster-{{ ds_nodash }}` (請參閱資訊方塊，瞭解選填的額外資訊)
- `num_workers` - 分配給 Cloud Dataproc 叢集的工作站數量

- `zone` - 我們希望叢集所在的地理區域，會儲存在 Airflow 資料庫中。這會讀取我們在步驟 3 中設定的 `'gce_zone'` 變數
- `master_machine_type` - 要分配給 Cloud Dataproc 主要執行個體的機器類型
- `worker_machine_type` - 要分配給 Cloud Dataproc 工作站的機器類型

`cluster_name` 變數的其他資訊

參數的 `{{ ds_nodash }}` 部分存在，是因為 Airflow 支援 [jinja2 範本](#)。這是 Airflow 在每次啟動運算子時，於執行階段轉譯的參數。在本例中，`{{ ds_nodash }}` 會換成 YYYYMMDD 格式的 DAG 執行日期。如果您不熟悉 jinja2 範本，也沒關係！完成本實驗室不需要具備相關知識。

提交 Apache Hadoop 工作

`dataproc_operator.DataProcHadoopOperator` 讓我們將工作提交至 Cloud Dataproc 叢集。

```
run_dataproc_hadoop = dataproc_operator.DataProcHadoopOperator(
    task_id='run_dataproc_hadoop',
    main_jar=WORDCOUNT_JAR,
    cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
    arguments=wordcount_args)
```

我們提供下列參數：

- `task_id` - 我們為這個 DAG 片段指派的名稱
- `main_jar` - 我們要在叢集上執行的 .jar 檔案位置
- `cluster_name` - 用來執行工作的叢集名稱，您會發現這與前一個運算子中的名稱相同
- `arguments` - 傳遞至 jar 檔案的引數，與從指令列執行 .jar 檔案時相同

刪除叢集

我們要建立的最後一個運算子是 `dataproc_operator.DataprocClusterDeleteOperator`。

```
delete_dataproc_cluster = dataproc_operator.DataprocClusterDeleteOperator(
    task_id='delete_dataproc_cluster',
    cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
    trigger_rule=trigger_rule.TriggerRule.ALL_DONE)
```

顧名思義，這個運算子會刪除指定的 Cloud Dataproc 叢集。這裡有三個引數：

- `task_id` - 與 `BashOperator` 相同，這是我們指派給運算子的名稱，可從 Airflow UI 查看
- `cluster_name`：我們指派給 Cloud Dataproc 叢集的名稱。我們將其命名為 `composer-hadoop-tutorial-cluster-{{ ds_nodash }}` (如需其他選用資訊，請參閱「建立 Dataproc 叢集」一節後的資訊方塊)
- `trigger_rule` - 我們在本步驟開頭的匯入作業中，簡要提及了觸發條件規則，但這裡有一個實際運作的規則。根據預設，除非所有上游運算子都順利完成，否則 Airflow 運算子不會執行。`ALL_DONE` 觸發規則只要求所有上游運算子都已完成，不論是否成功。這表示即使 Hadoop 工作失敗，我們仍要拆除叢集。

```
create_dataproc_cluster >> run_dataproc_hadoop >> delete_dataproc_cluster
```

最後，我們希望這些運算子依特定順序執行，因此可以使用 Python 位元移位運算子標示。在此情況下，系統一律會先執行 `create_dataproc_cluster`，再執行 `run_dataproc_hadoop`，最後執行 `delete_dataproc_cluster`。

整合所有內容後，程式碼如下所示：

```

# Copyright 2018 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

# [START composer_hadoop_tutorial]
"""Example Airflow DAG that creates a Cloud Dataproc cluster, runs the Hadoop
wordcount example, and deletes the cluster.

This DAG relies on three Airflow variables
https://airflow.apache.org/concepts.html#variables
* gcp_project - Google Cloud Project to use for the Cloud Dataproc cluster.
* gce_zone - Google Compute Engine zone where Cloud Dataproc cluster should be
  created.
* gcs_bucket - Google Cloud Storage bucket to use for result of Hadoop job.
  See https://cloud.google.com/storage/docs/creating-buckets for creating a
  bucket.
"""

import datetime
import os

from airflow import models
from airflow.contrib.operators import dataproc_operator
from airflow.utils import trigger_rule

# Output file for Cloud Dataproc job.
output_file = os.path.join(
    models.Variable.get('gcs_bucket'), 'wordcount',
    datetime.datetime.now().strftime('%Y%m%d-%H%M%S')) + os.sep
# Path to Hadoop wordcount example available on every Dataproc cluster.
WORDCOUNT_JAR = (
    'file:///usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar'
)
# Arguments to pass to Cloud Dataproc job.
input_file = 'gs://pub/shakespeare/rose.txt'

wordcount_args = ['wordcount', input_file, output_file]

yesterday = datetime.datetime.combine(
    datetime.datetime.today() - datetime.timedelta(1),
    datetime.datetime.min.time())

```

```

default_dag_args = {
    # Setting start date as yesterday starts the DAG immediately when it is
    # detected in the Cloud Storage bucket.
    'start_date': yesterday,
    # To email on failure or retry set 'email' arg to your email and enable
    # emailing here.
    'email_on_failure': False,
    'email_on_retry': False,
    # If a task fails, retry it once after waiting at least 5 minutes
    'retries': 1,
    'retry_delay': datetime.timedelta(minutes=5),
    'project_id': models.Variable.get('gcp_project')
}

# [START composer_hadoop_schedule]
with models.DAG(
    'composer_hadoop_tutorial',
    # Continue to run DAG once per day
    schedule_interval=datetime.timedelta(days=1),
    default_args=default_dag_args) as dag:
    # [END composer_hadoop_schedule]

    # Create a Cloud Dataproc cluster.
    create_dataproc_cluster = dataproc_operator.DataprocClusterCreateOperator(
        task_id='create_dataproc_cluster',
        # Give the cluster a unique name by appending the date scheduled.
        # See https://airflow.apache.org/code.html#default-variables
        cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
        num_workers=2,
        zone=models.Variable.get('gce_zone'),
        master_machine_type='n1-standard-1',
        worker_machine_type='n1-standard-1')

    # Run the Hadoop wordcount example installed on the Cloud Dataproc cluster
    # master node.
    run_dataproc_hadoop = dataproc_operator.DataProcHadoopOperator(
        task_id='run_dataproc_hadoop',
        main_jar=WORDCOUNT_JAR,
        cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
        arguments=wordcount_args)

    # Delete Cloud Dataproc cluster.
    delete_dataproc_cluster = dataproc_operator.DataprocClusterDeleteOperator(
        task_id='delete_dataproc_cluster',
        cluster_name='composer-hadoop-tutorial-cluster-{{ ds_nodash }}',
        # Setting trigger_rule to ALL_DONE causes the cluster to be deleted
        # even if the Dataproc job fails.
        trigger_rule=trigger_rule.TriggerRule.ALL_DONE)

    # [START composer_hadoop_steps]
    # Define DAG dependencies.
    create_dataproc_cluster >> run_dataproc_hadoop >> delete_dataproc_cluster

```



```
# [END composer_hadoop_steps]
```

```
# [END composer_hadoop]
```

如果您因為環境未建立而略過步驟 3，請務必先返回該步驟，再繼續執行步驟 5。

5. 將 Airflow 檔案上傳至 Cloud Storage

將 DAG 複製到 /dags 資料夾

1. 首先，請開啟 [Cloud Shell](#)，其中已安裝 Cloud SDK，方便您使用。
2. 複製 Python 範例存放區，然後變更為 composer/workflows 目錄

```
git clone https://github.com/GoogleCloudPlatform/python-docs-samples.git && cd
python-docs-samples/composer/workflows
```

3. 執行下列指令，將 DAG 資料夾名稱設為環境變數

```
DAGS_FOLDER=$(gcloud composer environments describe ${COMPOSER_INSTANCE} \
--location us-central1 --format="value(config.dagGcsPrefix)")
```

4. 執行下列 `gsutil` 指令，將教學課程程式碼複製到建立 /dags 資料夾的位置

```
gsutil cp hadoop_tutorial.py $DAGS_FOLDER
```

輸出內容應如下所示：

```
Copying file://hadoop_tutorial.py [Content-Type=text/x-python]...
/ [1 files][ 4.1 KiB/ 4.1 KiB]
Operation completed over 1 objects/4.1 KiB.
```

注意：本指南現在將工作流程稱為 `composer_hadoop_tutorial`

步驟 6 · 約 10 分鐘

6. 使用 Airflow UI

如要使用 GCP 控制台存取 Airflow 網頁介面：

1. 開啟「Environments」(環境) 頁面。
2. 在環境的「Airflow webserver」(Airflow 網路伺服器) 欄中，按一下新視窗圖示。Airflow 網頁使用者介面會在新瀏覽器視窗中開啟。



Composer

Environments

 CREATE

 DELETE



Filter environments

☐	 Name 	Location	Creation time	Update time	Airflow webserver
☐	 my-composer-environment	us-central1	3/8/19, 1:16 PM	3/8/19, 1:33 PM	 Airflow

如要進一步瞭解 Airflow 使用者介面，請參閱[存取網頁介面](#)。

查看變數

您先前設定的變數會保留在環境中。如要查看變數，請從 Airflow 使用者介面選單列中依序選取「Admin」>「Variables」。

Variables

Choose File

No file chosen

Import Variables

List (3)

Create

Add Filter▼

With selected▼

Search

☐		Key	Val
☐	 	gcp_project	project-id
☐	 	gcs_bucket	gs://bucket-name
☐	 	gce_zone	zone

瞭解 DAG 執行作業

將 DAG 檔案上傳到 Cloud Storage 中的 `dags` 資料夾後，Cloud Composer 會剖析檔案。如果沒有發現錯誤，工作流程名稱會顯示在 DAG 清單中，該工作流程會排入要立即執行的作業佇列。如要查看 DAG，請按一下頁面頂端的「DAG」。

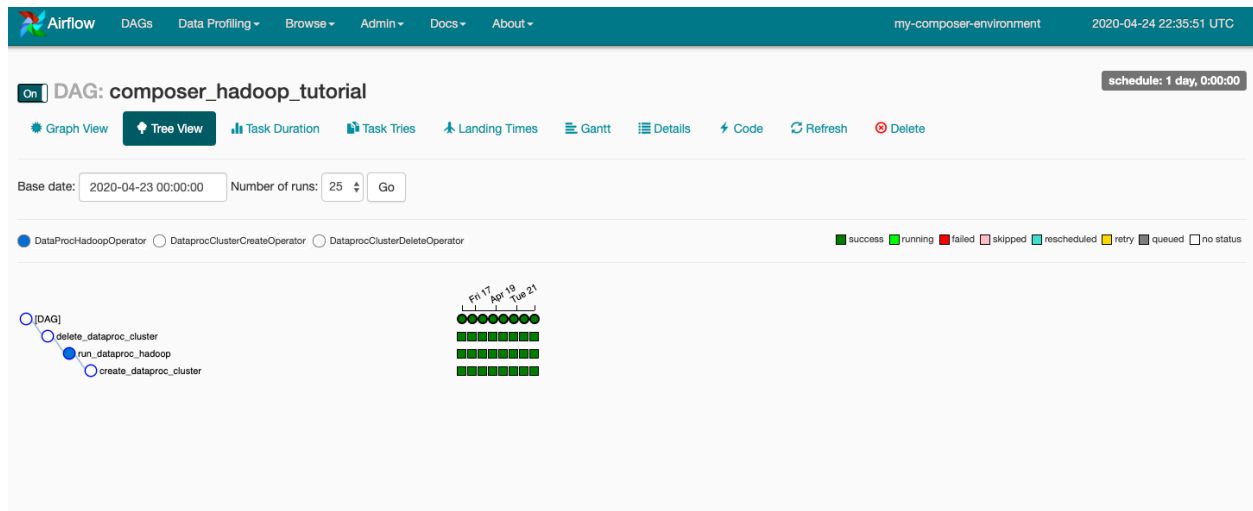
Airflow DAGs								
my-composer-environment 2020-04-24 22:35:23 UTC								
DAGs								
Search:								
		DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
	On	airflow_monitoring	None	Airflow		2020-04-24 22:32		
	On	composer_hadoop_tutorial	1 day, 0:00:00	Airflow		2020-04-23 00:00		

Showing 1 to 2 of 2 entries

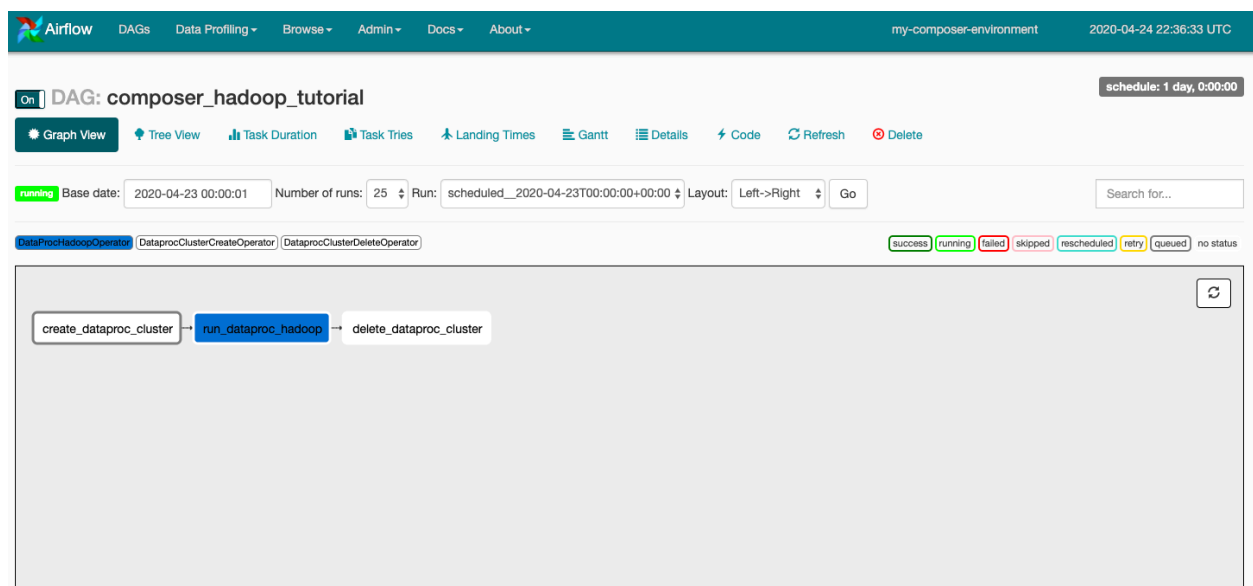
« < 1 > »

Hide Paused DAGs

按一下 `composer_hadoop_tutorial` 開啟 DAG 詳細資料頁面。此頁面會透過圖形呈現工作流程工作和依附元件。

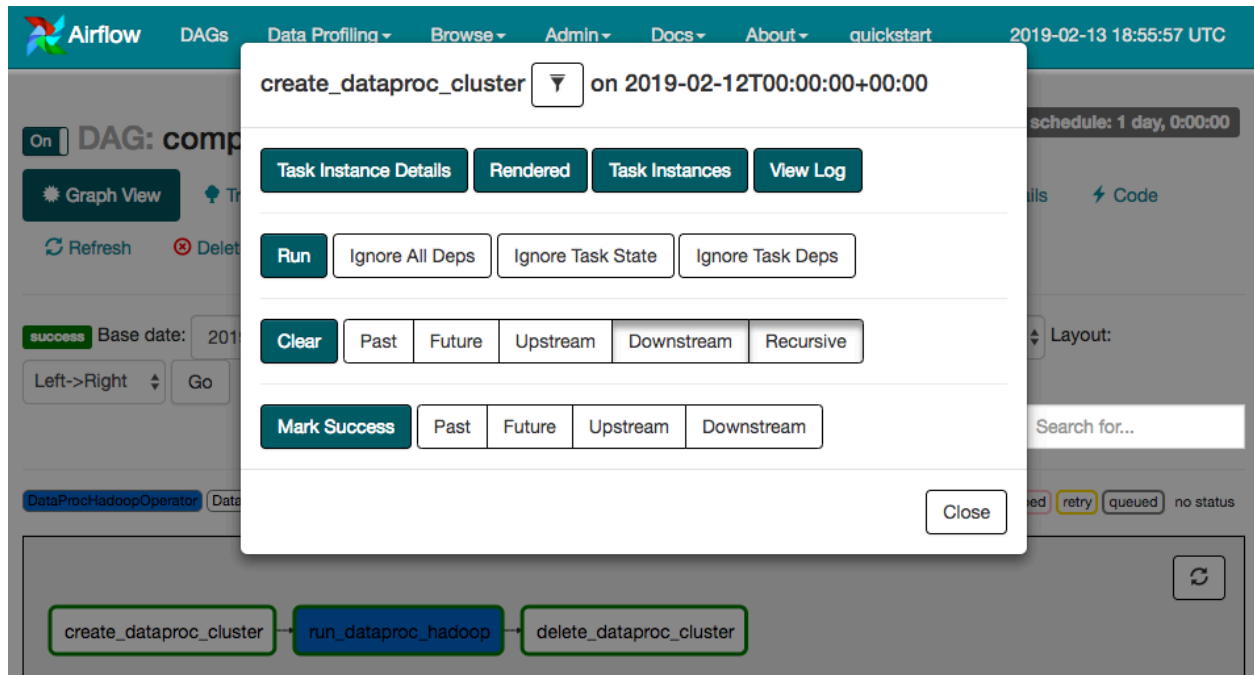


接著在工具列中點選「圖表檢視」，然後將滑鼠游標懸停在各工作的圖表上，即可查看工作狀態。請注意，每個工作周圍的框線也會指出狀態，例如綠色框線表示執行中，紅色框線表示執行失敗等。



如要再次從「圖表檢視畫面」執行工作流程：

1. 在 Airflow 使用者介面的「圖表」檢視畫面中，按一下 `create_dataproc_cluster` 圖示。
2. 按一下「清除」會重設這三項工作，點選「確定」即可確認操作。



您也可以前往下列 GCP Console 頁面，查看 `composer-hadoop-tutorial` 工作流程的狀態和結果：

- [Cloud Dataproc 叢集](#)，監控叢集的建立和刪除作業。請注意，工作流程建立的叢集是暫時性的，只會在工作流程執行期間存在，並在最後一個工作流程任務完成後刪除。
- [Cloud Dataproc Jobs](#)，即可查看或監控 Apache Hadoop 字數計算工作。按一下工作 ID 即可查看工作記錄輸出內容。
- [Cloud Storage 瀏覽器](#)，查看您為本程式碼研究室建立的 Cloud Storage bucket 中 `wordcount` 資料夾內的字數統計結果。

7. 清除所用資源

如要避免系統向您的 GCP 帳戶收取您在本程式碼研究室中所用資源的相關費用：

1. (選用) 如要儲存資料，請從 Cloud Composer 環境的 Cloud Storage bucket，以及您為本程式碼研究室建立的儲存空間 bucket [下載資料](#)。
2. 刪除您為本程式碼研究室建立的 [Cloud Storage bucket](#)。
3. 刪除環境的 [Cloud Storage bucket](#)。
4. 刪除 [Cloud Composer 環境](#)。請注意，刪除環境並不會刪除環境的儲存空間值區。

您也可以選擇刪除專案：

1. 前往 GCP 主控台的「Projects」(專案) 頁面。
2. 在專案清單中選取要刪除的專案，然後點按「Delete」(刪除)。
3. 在方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。

警告：刪除專案會出現以下結果：

- **專案中的所有內容都會遭到刪除。**如果您之前使用現有的專案來進行本教學課程，當您刪除該專案時，也會一併刪除您在該專案中完成的所有其他工作。
 - **自訂專案 ID 不復存在。**您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (例如 `appspot.com` 網址)，請刪除在該專案中選取的資源，而不是刪除整個專案。
-